Embedded Systems study plan

FONTYS ICT / FIRST YEAR / COACH: IMAN

A. Rationale - Why Embedded Systems?

I am considering Embedded Systems (Intelligent Technologies) because I enjoy C and understanding how software interacts with hardware. I have started using C, ESP32, Wi-Fi and MQTT in an IoT greenhouse project, and explored alternative communication protocols. This gives me a concrete system in which to investigate sensors, timing and communication decisions. My interest in registers, peripherals, interrupts and digital logic is a direction to investigate, not evidence of mastery. Embedded engineering requires reasoning about physical behaviour, memory, timing and failure as well as writing working code. After discussing this gap with Iman, I want to build the mathematics, electronics and computer science needed to explain why a design works and when another approach is better. This makes the specialisation a considered next step, which I will test through the evidence below.

B. Learning roadmap - 12 weeks

Scope: 12 weeks at 4.5 hours/week (54 hours). Selected lessons and small experiments, not complete courses. I assume access to my existing ESP32 development setup; no additional sensors, actuators or measuring instruments are assumed. Synthetic data and calculations are acceptable if clearly labelled.

Period	Learning objectives	Resource and reason for choosing it	Practical application	Deliverable / completion criteria
1-2	Functions, slope, units; voltage, current, resistance, power, series circuits, voltage dividers and capacitance. **Why:** turn sensor values and circuit choices into quantities I can justify.	OpenStax: College Algebra 2e, 3.1 & 4.1 for function examples; All About Circuits: DC, selected voltage/current, Ohm's law, divider and capacitor lessons for physical meaning. **Entry:** arithmetic and rearranging simple equations.	**W1:** diagnose algebra gaps; derive y = ax + b from two synthetic calibration points. **W2:** calculate divider output, current and power; sketch capacitor charging and explain stored charge.	One calculation sheet and labelled circuit. Check units and verify three calibration inputs, including one outside the fitted range. Explain divider loading and why an ideal calculation alone does not establish GPIO compatibility.
3-4	Binary/hex, unsigned bit masks, pointers, arrays, bounds, storage lifetime; CPU, RAM, flash and memory-mapped I/O. **Why:** understand how data and peripheral state become machine operations.	GNU C Language Manual: Integer Types, Bitwise Operations, Pointers, Arrays and Storage Class Specifiers. Direct explanations with small code examples. **Entry:** basic C variables, loops and functions; revise these first if needed.	**W3:** draw memory for a fixed array and pointer; trace a bit-mask update. **W4:** write a C function converting an array of synthetic sensor readings, passing its length explicitly. Avoid dynamic allocation.	Compiling program, memory sketch and normal/empty/boundary cases. Explain array bounds, pointer lifetime and unsigned wraparound; correctly predict a changed input before running it.
5-6	GPIO levels, Boolean masks, registers, timers, polling versus interrupts, sampling interval and jitter. Identify UART, I2C and SPI roles; implement only timing. **Why:** a correct reading also needs predictable timing.	Espressif: GPIO & RTC GPIO and GPTimer, selected configuration and callback examples. **Entry:** phase 2 and an existing build/flash workflow. Match docs to installed framework and chip; no framework migration.	**W5:** trace one GPIO setting from API to peripheral; draw timer-to-handler flow. **W6:** timestamp 100 periodic events with network activity off/on. Use the timer API available in my setup; defer printing/network work out of the ISR.	Annotated flow diagram, code and min/mean/max interval error. Explain polling, ISR work and observed jitter; distinguish measurements from a worst-case timing guarantee. **Fit review:** record one appealing task, one difficulty and one open question.

B. Learning roadmap - continued

Order: quantities and circuits > C and memory > timing > network reliability > calculus > feedback. Each phase receives 9 hours. Later work builds on the relevant completion gates.

Period	Learning objectives	Resource and reason for choosing it	Practical application	Deliverable / completion criteria
7-8	Wi-Fi link, IP addressing, TCP versus UDP; MQTT topics, QoS 0/1, retained state and reconnects; HTTP/CoAP trade-offs. **Why:** justify MQTT and distinguish delivery from freshness or successful actuation.	OASIS MQTT 3.1.1 sections 3.3 and 4.3; HTTP Semantics sections 3 and 9.3; CoAP RFC 7252 sections 2 and 4. Short primary-source excerpts for a defensible comparison. **Entry:** existing MQTT experiment and timing logs.	**W7:** draw MQTT/TCP/IP/Wi-Fi layers; compare publish/subscribe with request/response. **W8:** send numbered, timestamped readings, interrupt Wi-Fi and reconnect. Log loss, duplicates and stale data with the supported QoS setting.	Comparison table: pattern, transport, overhead assumptions, delivery and security; outage log with observed versus expected behaviour. Explain QoS 1 duplicates, retained-message age, bounded buffering and TLS/authentication. Name one scenario favouring HTTP and one favouring CoAP.
9-10	Derivatives as rate, integrals as accumulation; finite differences, sums and a first-order differential equation. **Why:** P, I and D need meanings and units before tuning makes sense.	3Blue1Brown: The paradox of the derivative and Integration and the fundamental theorem of calculus. Visual intuition, followed by my own calculations. **Entry:** phase 1 functions/units and phase 3 sampling; replay algebra work if these are weak.	**W9:** calculate temperature slopes and accumulated error for a short synthetic series. **W10:** step through dT/dt = (Tenv - T)/tau + k*u using Euler updates. State units; compare results at dt and dt/2.	Worked values and two response curves, with parameters labelled hypothetical. Explain tau, slope, area and discretisation error. **Gate:** compute difference/dt and sum(error*dt) unaided before starting PI control.
11-12	Feedback, setpoint/error, P and PI, saturation, windup, noise, sampling and signs of instability; interpret D only. **Why:** assess when feedback improves on simple on/off control.	MathWorks: Understanding PID Control, parts 1-3: PID basics, anti-windup and derivative noise. Conceptual videos; use my C model without MATLAB. **Entry:** phase 5 gate; full PID tuning and stability proofs are later work.	**W11:** reuse the thermal model; compare on/off with hysteresis against P. **W12:** add PI with output limits and conditional integration. Repeat with a disturbance and a larger sample interval. Predict each change first.	Overlaid curves and a table of peak overshoot, final error and settling time (state tolerance/horizon; report if not settled). Explain saturation, why D reacts to noise and why simulated stability is not proof for hardware. Final coach review.

C. Weekly routine

Theory - 2 h - Read/watch only the named extracts; solve or sketch before opening a worked answer.

Practice - 1.75 h - Run one controlled experiment: prediction, single change, recorded result.

Evidence - 0.75 h - Keep code, parameters, diagram/plot and a 150-word explanation: why, result, limitation, next question.

Total: 4.5 h/week. A fifth hour is optional catch-up, not new content. If a gate fails, use the next practice session to repair the gap and reduce later scope. If necessary, finish week 12 with P control and defer PI; do not compress the foundations.

D. Longer-term direction

Mathematics and electronics: extend algebra into trigonometry (phase and periodic signals), vectors/matrices (coordinates and coupled systems), then derivatives, integrals and differential equations in depth. Use these to analyse RC dynamics and sensor/actuator response; study logic gates and interface electrical limits.

Embedded and real-time systems: deepen architecture, registers, ADC/PWM and UART/I2C/SPI through one peripheral at a time. Then study RTOS scheduling, deadlines, task/ISR communication and race conditions. Demonstrate a measured timing budget and recovery from a failed peripheral.

Control engineering, then robotics: progress to modelling, transfer functions, stability analysis, sampling/aliasing, filtering and full PID with actuator constraints. After Linux CLI and C++ or Python basics, explore ROS2 nodes, topics, services and QoS using the official ROS2 tutorials (source). ROS2 supports communication between robotics components; it does not replace low-level control or guarantee deadlines. Start in simulation; choose an OS-compatible supported release at that time.

E. Progress evaluation

1. **Explain:** at each fortnightly checkpoint, give a five-minute explanation without tutorial notes: principle, implementation, alternative and limitation. Correct gaps before moving on.

2. **Predict and transfer:** make at least one written prediction per phase and solve one changed-input task without copying a solution. Explain any mismatch using units, code or a model.

3. **Reproduce:** keep six evidence bundles with source links, code/version, inputs, assumptions and results. Re-run one experiment with Iman from its recorded steps.

4. **Reason about failure:** by week 12, show evidence for an array-boundary case, a network outage and actuator saturation in simulation; distinguish observations from guarantees.

5. **Decide on fit:** review at weeks 6 and 12. Use two concrete examples of enjoying or disliking hardware/software reasoning, one persistent gap and the workload log to decide: continue, extend foundations or investigate another specialisation.

Resource access and prerequisites

Resource titles, linked content and relevance checked on 29 October 2026. The core readings/video pages are publicly accessible; no paid course enrolment or certificate is required. Study only the named sections. Entry requirements in the table are my study readiness checks, not provider-issued admissions requirements. GNU material includes extensions: focus on portable C and verify target-specific details. Espressif API versions vary; select the installed framework/chip version. The ROS2 rendered documentation blocked automated access, so its official source was verified instead.